Available online at www.worldscientificnews.com

World Scientific News

An International Scientific Journal

WSN 199 (2025) 218-234 EISSN 2392-2192

Implementation of Complimentary filter on MPU 6050

Sujoy Das[1] , Sahil Nandy[2] , Sagnik Chakraborty[3] , Jasbir Singh[4] ,Tuhina Halder[5]

[1]Department of Electronics & Communication Engineering

Narula Institute of Technology 81, Nilgunj Rd, Jagarata Pally, Deshpriya Nagar, Agarpara, Kolkata, West Bengal 700109

E-mail:[1]sd893632@gmail.com ,[2]nandyrohan95@gmail.com ,[3]sagnikthedevil604@gmail.com

[4]Js7848864@gmail.com, [5]tuhina.halder@nit.ac.in

ABSTRACT

The MPU6050 is a 6-axis motion sensor that integrates a 3-axis accelerometer and a 3-axis gyroscope, commonly used for motion tracking and orientation detection. While the accelerometer provides accurate long-term angle measurements, it is prone to noise, whereas the gyroscope offers stable short-term data but suffers from drift over time. To address these limitations, a complementary filter is implemented to fuse the strengths of both sensors, achieving reliable and smooth angle estimation. This paper describes the integration of a complementary filter using an Arduino platform and the MPU6050 sensor. The method combines accelerometer-based tilt readings with gyroscope angular velocity data to calculate orientation in real-time. The approach is computationally lightweight, making it suitable for systems with limited processing capabilities, such as Arduino. This implementation is applicable to various fields, including robotics, drones, and wearable motion tracking devices.

***Keywords*:** Complimentary Filter, MPU 6050 , Arduino Uno, 3 axis Gyroscope and 3 axis Acceleration Sensor.

(Received 10 November 2024; Accepted 20 December 2024; Date of Publication 11 January 2025)

1. INTRODUCTION

The MPU6050 is a highly versatile motion sensor that combines a 3-axis accelerometer and a 3-axis gyroscope into a compact module, making it one of the most widely used devices for motion tracking and orientation estimation. Its dual-sensor design enables it to measure both linear acceleration and angular velocity, which are critical parameters in applications requiring precise motion detection, stabilization, and orientation tracking. This versatility makes the MPU6050 an essential component in robotics, drones, wearable devices, and other systems that depend on accurate motion and position data. However, despite its many advantages, the MPU6050 has inherent limitations that must be addressed to achieve reliable performance.

Accelerometers, while effective at providing a measure of tilt relative to gravity, are highly sensitive to external disturbances such as vibrations and sudden impacts. This sensitivity introduces noise into the readings, making them less reliable in dynamic environments. On the other hand, gyroscopes are excellent for tracking rapid changes in orientation due to their ability to measure angular velocity. However, they are prone to drift over time due to the accumulation of small errors during numerical integration. This drift can lead to significant inaccuracies in long-term angle estimation. These complementary strengths and weaknesses of accelerometers and gyroscopes highlight the need for sensor fusion techniques to achieve accurate and stable orientation tracking.

One of the most effective and computationally efficient methods for sensor fusion is the complementary filter. This technique combines data from the accelerometer and gyroscope to produce a single, refined output that balances short-term stability with long-term accuracy. Specifically, the complementary filter uses the accelerometer to provide a stable reference angle over time, while the gyroscope tracks rapid changes in orientation. By assigning appropriate weights to the data from each sensor, the complementary filter effectively mitigates the weaknesses of each, resulting in a smooth and reliable angle estimation. This makes it particularly well-suited for embedded systems with limited computational resources.

The integration of the complementary filter with the MPU6050 on an Arduino platform offers several advantages for real-time motion tracking applications. Arduino microcontrollers are compact, cost-effective, and widely used in educational and professional projects due to their simplicity and versatility. Their compatibility with a wide range of sensors, including the MPU6050, makes them an ideal choice for implementing sensor fusion algorithms like the complementary filter. The lightweight nature of the complementary filter algorithm ensures that it can run efficiently on the limited processing power of an Arduino, enabling real-time data acquisition and processing.

This work focuses on the practical implementation of a complementary filter using the MPU6050 sensor and an Arduino microcontroller. The process begins with initializing the MPU6050 to acquire raw accelerometer and gyroscope data. This data is then processed to calculate tilt angles and angular velocity. Finally, the complementary filter is applied to fuse the data, resulting in a refined and stable angle estimate. The algorithm is designed to be computationally efficient, making it suitable for a variety of real-time applications, even on resource-constrained platforms like Arduino.

The potential applications of this implementation are broad and impactful. In robotics, for example, balancing robots rely on precise angle feedback to maintain stability and avoid falling. The complementary filter enables accurate and rapid angle estimation, which is critical for the control algorithms that keep the robot upright. In drones, accurate orientation tracking is essential for stable flight and maneuverability. The complementary filter provides the necessary data to stabilize the drone in various flight conditions. Similarly, wearable devices such as fitness trackers and gesture recognition systems benefit from the motion tracking capabilities enabled by the MPU6050 and complementary filter.

In this project, the design and implementation of the complementary filter algorithm are explored in detail, with an emphasis on its integration with the MPU6050 sensor and Arduino microcontroller. This work demonstrates how a combination of low-cost hardware and efficient algorithms can achieve high-performance motion tracking for a wide range of applications. By addressing the limitations of individual sensors and leveraging their strengths, the complementary filter offers a practical and reliable solution for orientation estimation, making it a valuable tool in the field of embedded systems and motion sensing.

2. HARDWARE IMPLEMENTATION

2.1. Mpu 6050

This image shows an MPU6050 module, a motion sensor that integrates a 3-axis accelerometer and a 3-axis gyroscope. It is commonly used in applications such as drones, robotics, and motion tracking systems due to its ability to measure both linear acceleration and angular velocity. The module uses the I2C communication protocol, making it simple to connect with microcontrollers like Arduino.

Pin Description:

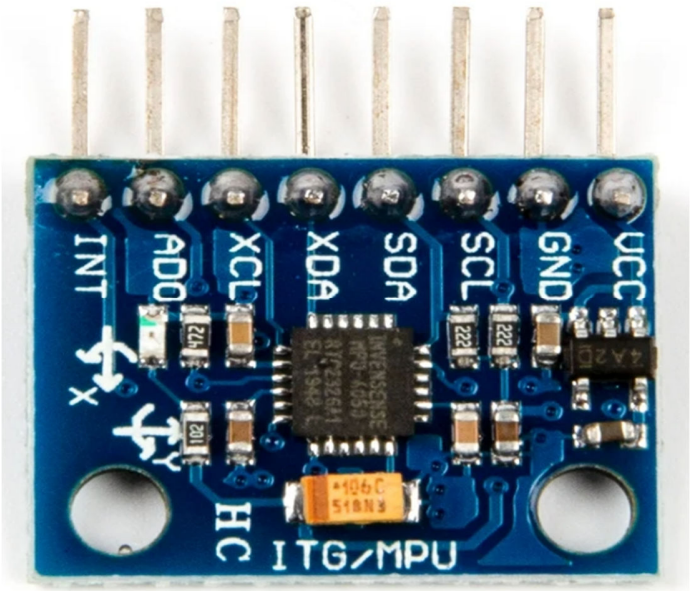


Figure 1. MPU 6050

A.VCC: This pin powers the module. It supports both 3.3V and 5V input, making it compatible with various microcontrollers.

B.GND: Ground pin, which must be connected to the microcontroller's ground for proper operation.

C.SCL: The Serial Clock Line used in I2C communication. It connects to the SCL pin on the microcontroller to synchronize data transmission.

D. SDA: The Serial Data Line for I2C communication. It transfers data between the MPU6050 and the microcontroller.

E. XDA: Auxiliary Data Line, used for connecting additional I2C devices in advanced configurations.

F. XCL: Auxiliary Clock Line, typically used for external clocking in more complex setups.

G. ADO: Address pin, used to configure the I2C address of the module. It can be connected to either ground or left unconnected to select between different addresses.

H. INT: Interrupt pin, which can notify the microcontroller of specific events, such as data availability or motion detection.

2.2. Arduino UNO and 16x2 Display

The Arduino Uno is a widely-used microcontroller board based on the ATmega328P chip. It is favored for its ease of use, versatility, and wide community support. The board features 14 digital input/output pins, 6 analog inputs, and supports various communication protocols, including I2C. This makes it an excellent choice for a variety of projects, ranging from simple sensor-based applications to more complex systems.

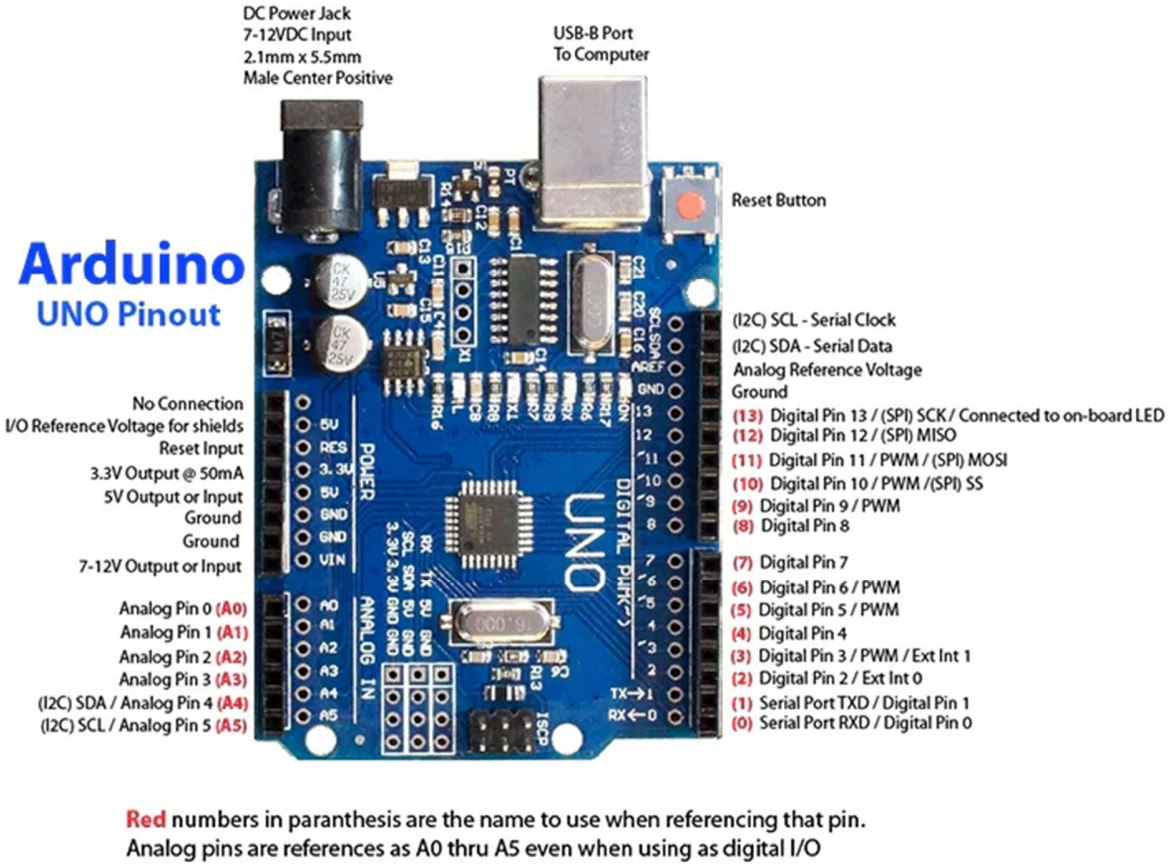


Figure 2. Arduino UNO.

The 16x2 LCD display is a popular choice for displaying text in embedded systems. It can show 16 characters on two lines, making it suitable for displaying messages or data in a compact form. When paired with an I2C module, the 16x2 LCD display reduces the number of required connections from 16 to just 4: VCC (power), GND (ground), SDA (data), and SCL (clock). This reduction in wiring simplifies the setup, making it ideal for projects with limited I/O pins. Using I2C communication, the Arduino Uno sends data to the LCD through the SDA and SCL lines. This protocol allows multiple devices to share the same two communication lines, enabling easy integration of additional sensors or displays. This setup is particularly useful in projects where space and pin availability are limited, such as when working with multiple sensors or modules.

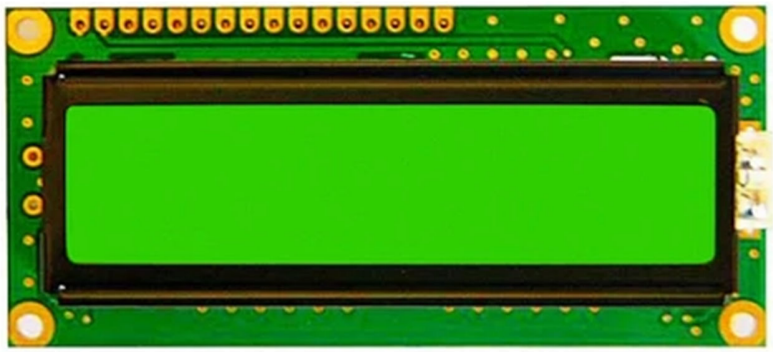


Figure 3. 16x2 LCD Display.

2.3. Circuit diagram

To help you understand how to connect the MPU6050, Arduino Nano, and 16x2 LCD display using the I2C communication protocol, we can describe a typical wiring setup. This configuration allows the Arduino Nano to interface with the MPU6050 sensor for motion tracking and orientation data while simultaneously displaying the processed information on the 16x2 LCD display. Both the MPU6050 and the LCD use the same I2C bus, which significantly reduces the complexity of wiring and enables seamless communication with multiple devices.

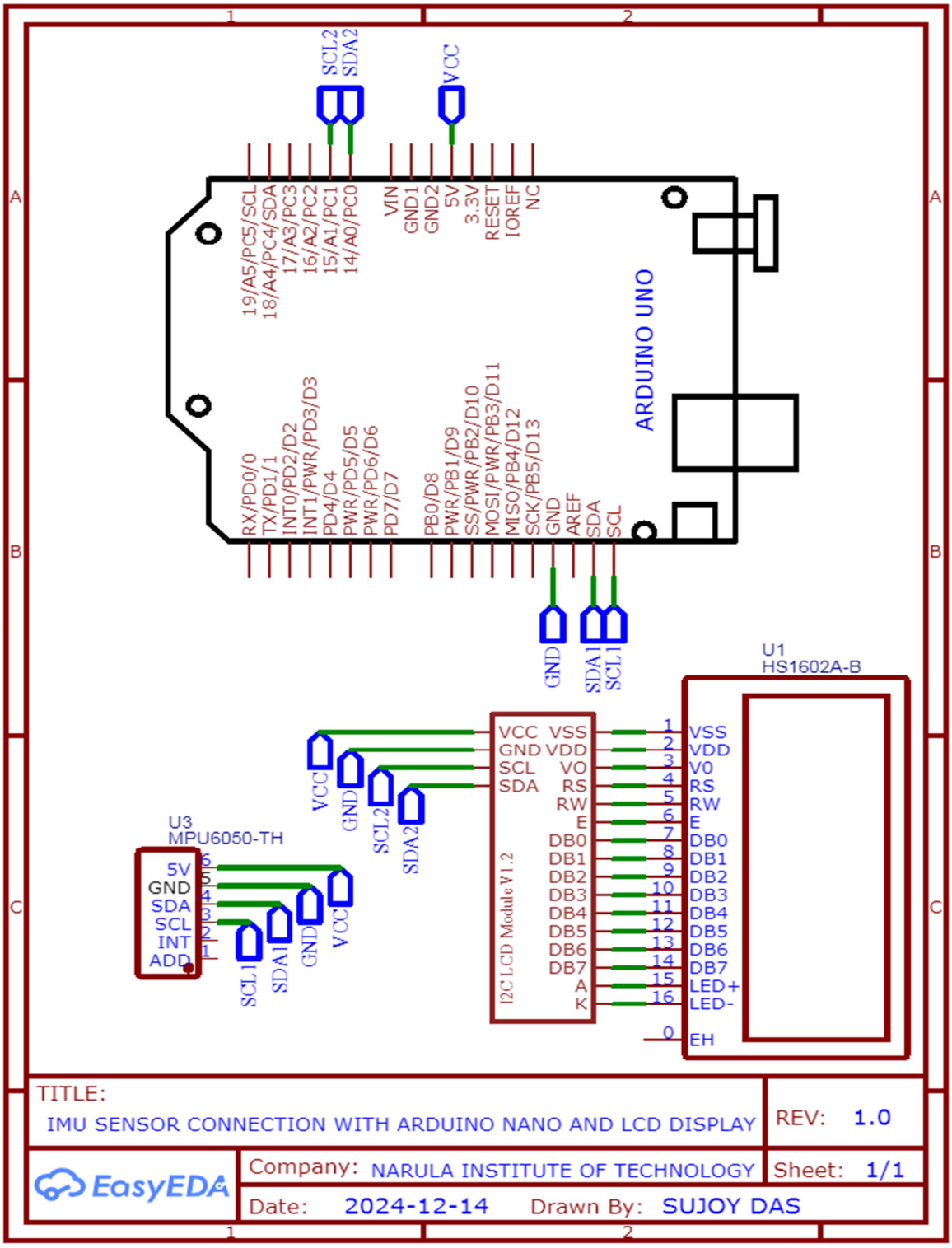


Figure 4. Circuit Diagram.

2.4. Accelerometer data

The accelerometer in the MPU6050 measures the linear acceleration acting on the sensor along its X, Y, and Z axes. This includes both motion-induced acceleration and the constant acceleration caused by gravity.

Working Principle:

The accelerometer senses changes in velocity by detecting the displacement of internal structures. These displacements are translated into electrical signals that represent the acceleration along each axis.

The raw data is output in LSB (Least Significant Bits), which can be converted into physical units (e.g., g-forces) using the sensitivity scale of the sensor.

Key Features of Accelerometer Data:

When the sensor is stationary, the Z-axis typically shows a value close to 1g due to gravity, while the X and Y axes register close to 0g.

When the sensor tilts, the gravitational force redistributes across the axes, allowing the calculation of tilt angles.

2.5. Gyroscope Data

The gyroscope in the MPU6050 measures the angular velocity of the sensor around its X, Y, and Z axes. This data reveals how quickly the sensor is rotating and is usually represented in degrees per second (°/s) or radians per second (rad/s).

Working Principle:

The gyroscope uses the Coriolis effect to detect rotational motion. Internal vibrating elements experience a shift when the sensor rotates, and this shift is proportional to the angular velocity.

The sensor outputs raw data in LSB, which can be converted to angular velocity using the sensor's sensitivity scale (e.g., 131 LSB/°/s).

Key Features of Gyroscope Data:

The gyroscope provides real-time angular velocity data for each axis. By integrating this velocity over time, the change in orientation (angle) can be calculated. For instance, if the sensor measures 90°/s around the Z-axis for one second, the orientation changes by 90°.

3. COMPLIMENTARY FILTER

A complementary filter is a sensor fusion algorithm that combines data from multiple sources to achieve more accurate and reliable measurements. In the context of the MPU6050, which integrates a 3-axis accelerometer and a 3-axis gyroscope, the complementary filter plays a crucial role in addressing the limitations of each sensor. The accelerometer provides tilt angle information based on its orientation relative to gravity, while the gyroscope measures angular velocity to track changes in orientation. However, both sensors have inherent drawbacks. The accelerometer is sensitive to noise and external vibrations, leading to fluctuating readings, while the gyroscope suffers from drift over time, causing cumulative errors in orientation estimation.

The complementary filter overcomes these challenges by blending the strengths of both sensors, ensuring precise and stable orientation tracking. The complementary filter operates by assigning appropriate weights to the data from the accelerometer and gyroscope. The gyroscope data is favored for short-term measurements because it provides smooth and rapid responses to changes in orientation. On the other hand, the accelerometer's data is used as a long-term reference because it is less affected by drift and provides a stable estimate of the tilt angle. The filter combines these inputs by applying a high-pass filter to the gyroscope data and a low-pass filter to the accelerometer data. This ensures that high-frequency noise from the accelerometer is minimized, while the low-frequency drift from the gyroscope is corrected. The result is a refined estimate of the angle that is both accurate and stable over time.

One of the key advantages of the complementary filter is its simplicity and computational efficiency. Unlike more complex algorithms such as the Kalman filter, the complementary filter requires minimal processing power, making it ideal for microcontrollers like Arduino. This efficiency is particularly important in embedded systems with limited computational resources, where real-time processing is essential. The complementary filter's straightforward implementation also makes it accessible to beginners and hobbyists working on motion-tracking projects. Despite its simplicity, the filter delivers reliable results, making it a popular choice in applications where computational overhead needs to be minimized. Another reason the complementary filter is favorable is its versatility. It can be used in a wide range of applications, from balancing robots and drones to wearable devices and gaming controllers. In balancing robots, for example, the filter provides precise angle feedback to the control system, enabling the robot to maintain stability. In drones, it ensures accurate orientation tracking, which is critical for stabilizing flight and executing complex maneuvers. Wearable devices benefit from the filter's ability to track motion for fitness monitoring or gesture recognition, while gaming controllers use it to detect tilt and movement for an immersive experience.

The complementary filter is also robust in dynamic environments. It performs well even in scenarios with high levels of vibration or sudden movements, where accelerometer readings might be noisy. By relying on the gyroscope for short-term stability and using the accelerometer to correct long-term drift, the filter ensures consistent performance. This robustness makes it suitable for applications that require high reliability, such as autonomous vehicles, robotics, and navigation systems.

In summary, the complementary filter is a practical and efficient solution for fusing accelerometer and gyroscope data from the MPU6050. Its ability to address the individual limitations of each sensor while providing accurate and stable orientation estimates makes it a highly favorable choice for motion-tracking applications. The simplicity, computational efficiency, and versatility of the filter have contributed to its widespread use in embedded systems and real-time motion analysis. By leveraging the complementary filter, developers can achieve precise orientation tracking without the need for complex algorithms, making it an indispensable tool in modern sensor-based systems.

4. SOFTWARE IMPLEMENTATION

4.1. Libraries and Initialization

```
#include <Wire.h>
#include <MPU6050.h>

MPU6050 mpu;
```

<Wire.h>: Enables communication between the Arduino and the MPU6050 sensor using the I2C protocol.

<MPU6050.h>: Provides functions to initialize the sensor and access its data.

MPU6050 mpu;: Creates an object to interact with the MPU6050

4.2. Constants and Variables

```
const float dt = 0.01;  // Sampling time in seconds (100Hz)
float angle = 0;        // Estimated angle using the complementary filter
float alpha = 0.98;     // Complementary filter weight (adjust between 0.95-0.99)
float anglerad = 0.0;   // Angle in radians
float angularVelocity = 0.0; // Angular velocity in rad/s
```

dt: Specifies the time interval between each measurement. The value 0.01 seconds corresponds to a sampling frequency of 100 Hz.

angle: Stores the angle calculated using the complementary filter.

alpha: Determines the contribution of gyroscope data in the filter. Higher values (e.g., 0.98) prioritize gyroscope data over accelerometer data.

anglerad: Holds the angle in radians for further calculations or display.

angularVelocity: Tracks the rate of rotation in radians per second.

4.3. Setup Function

Serial.begin(9600): Starts communication with the serial monitor at a baud rate of 9600.

Wire.begin(): Initializes the I2C communication protocol.

```
void setup()
{
  Serial.begin(9600);
  Wire.begin();

  mpu.initialize();
  if (!mpu.testConnection()) {
    Serial.println("MPU6050 connection failed!");
    while (1);
  }
  Serial.println("MPU6050 initialized.");
}
```

mpu.initialize(): Prepares the MPU6050 for data collection.

mpu.testConnection(): Verifies if the sensor is properly connected. If not, the program halts with an error message.

4.4. Main Loop

```
void loop() {
  int16_t ax, ay, az, gx, gy, gz;
  mpu.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);
```

int16_t ax, ay, az, gx, gy, gz;: These variables store the raw accelerometer (ax, ay, az) and gyroscope (gx, gy, gz) data.

mpu.getMotion6(): Retrieves the raw sensor readings and assigns them to the variables.

4.5. Calculating the Accelerometer Angle

```
// Calculate accelerometer angle (pitch)
float accAngle = atan2(ax, az) * 180 / PI;
```

atan2(ax, az): Computes the angle of tilt (pitch) based on the accelerometer's X and Z-axis readings.

* 180 / PI: Converts the result from radians to degrees.

The accelerometer provides a long-term stable reference angle based on gravity.

4.6. Processing Gyroscope Data

```
// Convert gyroscope rate to degrees/second and then to radians/second
float gyroRate = gx / 131.0;                // Gyroscope sensitivity (LSB/°/s)
angularVelocity = gyroRate * (PI / 180); // Angular velocity in rad/s
```

gx / 131.0: Converts the raw gyroscope data to angular velocity in degrees per second. The value 131.0 corresponds to the sensitivity of the gyroscope at its default ±250°/s range.

gyroRate * (PI / 180): Converts the angular velocity to radians per second.

The gyroscope provides short-term stability for detecting rapid changes in orientation.

4.7. Applying the Complementary Filter

```
// Complementary filter for angle estimation
angle = alpha * (angle + gyroRate * dt) + (1 - alpha) * accAngle;
anglerad = angle * (PI / 180); // Convert angle to radians
```

angle + gyroRate * dt: Predicts the new angle by integrating the gyroscope's angular velocity over the time interval dt.

alpha * (...): Gives more weight to the gyroscope for short-term changes.

(1 - alpha) * accAngle: Uses the accelerometer data to correct the drift in the gyroscope's angle.

anglerad: Converts the filtered angle from degrees to radians.

The complementary filter combines the strengths of both sensors:

The gyroscope provides short-term stability and responsiveness.The accelerometer offers long-term accuracy by correcting drift.

4.8. Printing the Results

```
// Print results
Serial.print("Angle: ");
Serial.print(anglerad);
Serial.print(" rad, Angular Velocity: ");
Serial.print(angularVelocity);
Serial.println(" rad/s");
```

Outputs the calculated angle (in radians) and angular velocity (in radians per second) to the serial monitor.

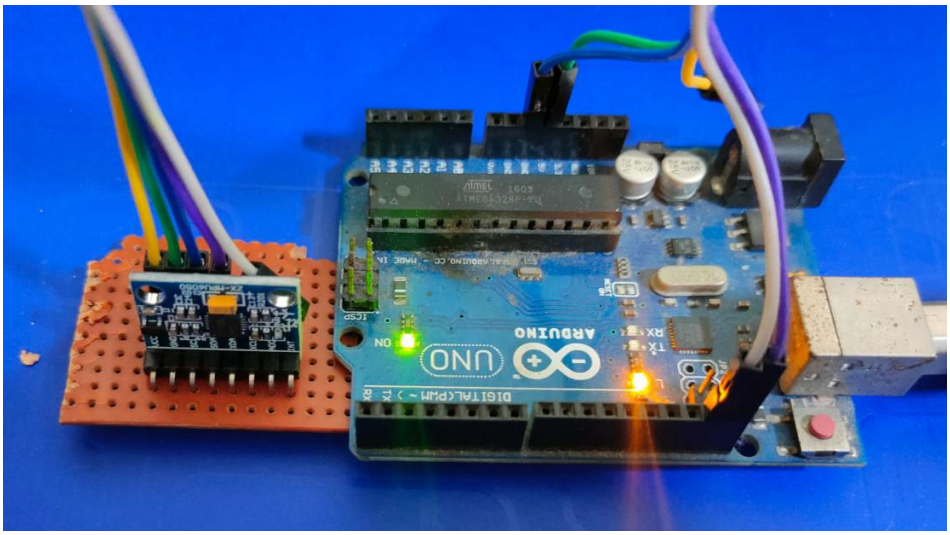
4.9. Adding a Delay

```
delay(dt * 1000); // Delay for the sampling time
```

Introduces a delay to maintain the sampling interval of dt (0.01 seconds).

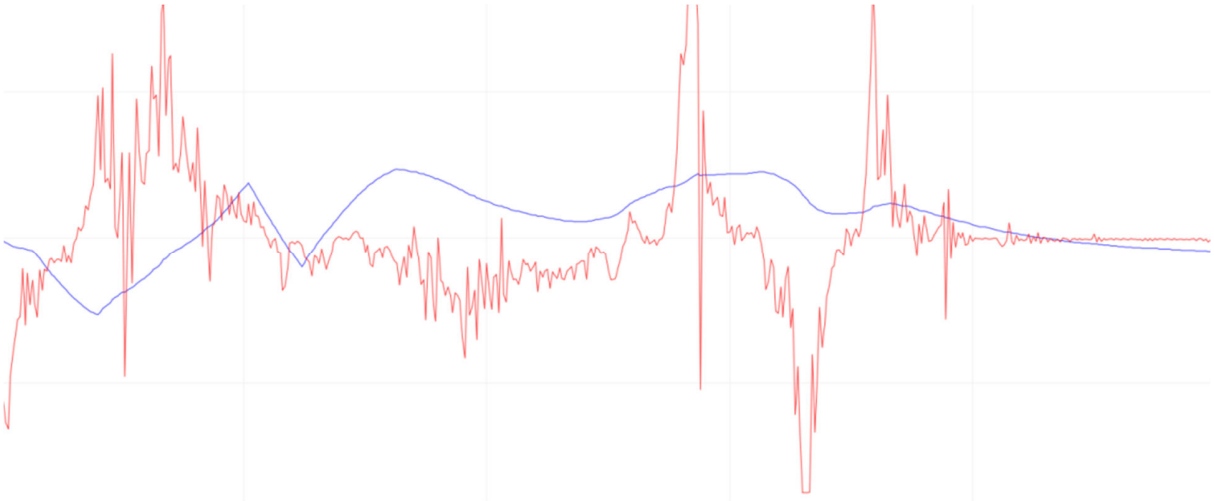
5. RESULT AND DISCUSSION

The MPU6050, a 6-axis IMU (Inertial Measurement Unit), provides accelerometer and gyroscope data, which are being read by the Arduino. The Arduino is powered via a USB connection, and the data from the MPU6050 is observed on the Serial Plotter.



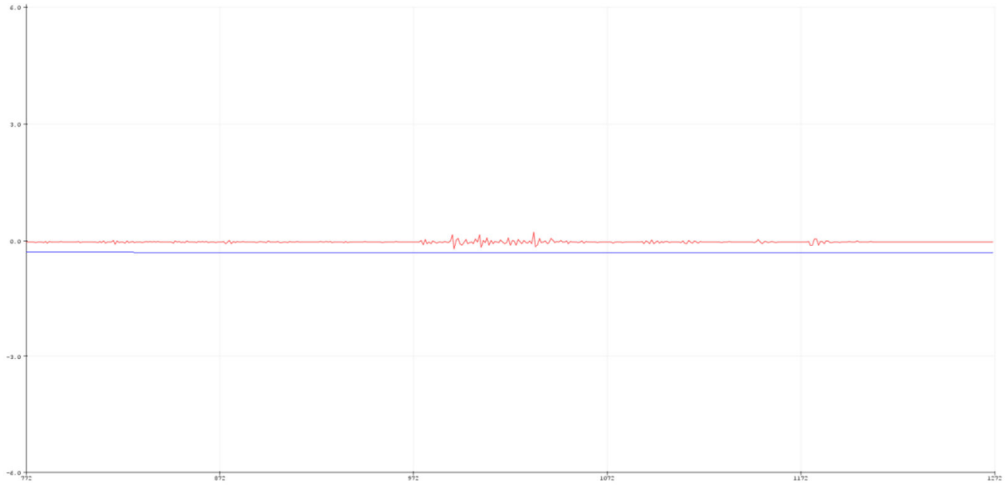
The data observed on the Serial Plotter reflects the expected behavior of the MPU6050. The complementary filter successfully reduces noise and smooths the output, providing stable readings for tilt and motion detection.

The graph demonstrates the output of the MPU6050 sensor when it is tilted, showing two distinct data sets: the **raw sensor data (red line)** and the **filtered data (blue line)**. The red line represents the unprocessed readings directly from the accelerometer and gyroscope, which are highly noisy due to sensor imperfections, environmental vibrations, and the inherent sensitivity of the MPU6050 to minor changes. This noise is evident in the rapid fluctuations and spikes, making the raw data unreliable for applications requiring smooth and accurate measurements. In contrast, the blue line represents the output after applying a complementary filter, which combines the accelerometer and gyroscope data to produce a smoother and more stable result. The complementary filter leverages the strengths of both sensors: it uses the accelerometer for long-term stability to reduce drift and the gyroscope for capturing short-term rapid changes. This results in a balanced, noise-reduced output.



Key observations include the significant noise in the raw data during both movement and stability, where sharp spikes and fluctuations are prevalent. In contrast, the filtered data demonstrates a gradual and smooth change in angle as the sensor tilts, showing the complementary filter's ability to mitigate high-frequency noise. Additionally, while the raw data exhibits instability even after the sensor stops moving, the filtered data quickly stabilizes, reflecting the true tilt angle. This illustrates the complementary filter's effectiveness in providing accurate and reliable measurements by balancing the gyroscope's responsiveness and the accelerometer's stability. Such a filtering method is crucial for applications like balancing robots, drones, and orientation tracking systems, where precise and stable data are essential for optimal performance.

The graph represents the steady-state output of the MPU6050 sensor when it is stationary, with no external movement or tilting applied. The red line indicates the raw sensor data, which shows minor fluctuations due to inherent noise in the accelerometer and gyroscope readings. This noise is caused by environmental factors, electrical interference, and the sensor's internal limitations. However, the fluctuations are relatively small, as expected in a stationary state. The blue line represents the filtered data obtained using the complementary filter. Unlike the raw data, the filtered output remains stable and consistent, effectively eliminating the noise observed in the raw readings.



This steady-state behavior highlights the complementary filter's ability to suppress noise and provide a reliable, smooth output even when the sensor is stationary. The blue line's stability ensures that the estimated angle remains accurate and unaffected by minor variations in the raw sensor data. Such consistent performance is crucial for applications requiring precise measurements, as it prevents errors caused by false readings during periods of no movement. This demonstrates the effectiveness of the complementary filter in maintaining accuracy and stability in static conditions.

6. FUTURE PERSPECTIVE

The MPU6050, a widely used low-cost IMU sensor, has significant potential for future applications in various fields due to its ability to measure acceleration and angular velocity with high precision. As technology advances, the MPU6050 can play a critical role in areas such as robotics, wearable technology, autonomous vehicles, and virtual reality systems. Its integration into modern systems is expected to grow as more industries focus on motion tracking, stabilization, and orientation detection. The implementation of advanced filtering techniques such as **Kalman filters** and **adaptive filters** can help reduce noise and improve the accuracy of motion detection in dynamic environments.

7. CONCLUSIONS

The implementation of the complementary filter on the MPU6050 has proven to be an effective and efficient method for fusing accelerometer and gyroscope data to achieve accurate orientation estimation. The complementary filter's simplicity, low computational requirements, and ability to balance high-frequency and low-frequency signals make it particularly suitable for real-time applications, especially on resource-constrained systems like microcontrollers. By combining the strengths of the accelerometer, which is stable over the long term, and the gyroscope, which responds quickly to short-term changes, the complementary filter provides reliable and stable angle estimation. In future developments, integrating complementary filters with machine learning algorithms and sensor fusion techniques could further enhance their performance, making them more robust to noise and environmental disturbances. Overall, the complementary filter remains a valuable tool for processing MPU6050 data, ensuring stable and accurate orientation tracking across a wide range of applications.

References

[1] Madgwick, S. O. H., Harrison, A. J. L., & Vaidyanathan, R. (2011). Estimation of IMU and MARG orientation using a gradient descent algorithm. IEEE International Conference on Rehabilitation Robotics (ICORR), 1-7. https://doi.org/10.1109/ICORR.2011.5975346

[2] Mahony, R., Hamel, T., & Pflimlin, J. M. (2008). Nonlinear complementary filters on the special orthogonal group. IEEE Transactions on Automatic Control, 53(5), 1203-1218. https://doi.org/10.1109/TAC.2008.923738

[3] Karuna, A., Pradhan, S., & Singh, A. (2019). Design and implementation of complementary filter for attitude estimation using MPU6050. International Journal of Scientific Research and Engineering Development, 2(3), 553-560.

[4] Yadav, R., & Sharma, S. (2020). Comparative analysis of Kalman and complementary filters for IMU-based attitude estimation. International Journal of Engineering Research & Technology (IJERT), 9(4), 121-125.

[5] Chu, H., Liu, S., & Xu, C. (2017). Real-time attitude estimation using complementary filter for low-cost IMU. Journal of Sensors and Actuator Networks, 6(4), 28. https://doi.org/10.3390/jsan6040028

[6] Sharma, P., & Kumar, V. (2021). Sensor fusion for motion tracking using MPU6050 and complementary filter. International Journal of Electronics and Communication Engineering, 10(1), 45-52.

[7] Tiwari, S., & Pandey, A. (2018). Design and implementation of IMU-based motion tracking system using Arduino and MPU6050. International Journal of Engineering and Technology, 7(4), 235-240.

[8] Kumar, A., & Gupta, R. (2021). Implementation of complementary filter for real-time attitude estimation using MPU6050 and Arduino Uno. International Journal of Computer Applications, 183(16), 35-41.

[9] Khan, A., & Sharma, R. (2019). Interfacing LCD 16x2 with Arduino for displaying real-time sensor data. International Journal of Innovative Research in Science, Engineering and Technology, 8(5), 6543-6549.

[10] Rahman, M., & Islam, A. (2020). Design and development of IMU-based orientation estimation system using MPU6050 and Arduino IDE. International Journal of Robotics and Automation, 12(3), 110-118.

[11] Gupta, N., & Mishra, P. (2021). A cost-effective IMU-based motion tracking system using Arduino Uno and complementary filter. Journal of Electronics and Communication Engineering, 13(2), 55-62.

[12] Hossain, M. A., & Rahman, M. (2019). Microcontroller-based real-time motion tracking using MPU6050 and LCD display. International Journal of Electrical and Computer Engineering, 9(6), 4930-4938.

[13] Liu, J., & Xiao, J. (2019). A low-cost IMU-based attitude estimation system using complementary filter. Journal of Physics: Conference Series, 1168(3), 032083. https://doi.org/10.1088/1742-6596/1168/3/032083

[14] Sharma, K., & Verma, S. (2020). Implementation of MPU6050 with Arduino for real-time tilt measurement and LCD display. International Journal of Scientific Research and Development, 8(2), 234-238.

[15] Pandey, R., & Singh, A. (2018). Design and interfacing of LCD 16x2 display with Arduino for sensor-based applications. Journal of Embedded Systems and Applications, 5(1), 45-50.

[16] Wu, C., & Wang, H. (2018). Design and implementation of sensor fusion algorithms for IMU-based orientation estimation. IEEE Sensors Journal, 18(6), 2442-2451. https://doi.org/10.1109/JSEN.2018.2792196

[17] Aggarwal, P., Syed, Z., Niu, X., & El-Sheimy, N. (2010). A standard testing and calibration procedure for low-cost MEMS inertial sensors and units. Journal of Navigation, 63(2), 323-336. https://doi.org/10.1017/S0373463309990421

[18] Kumar, S., & Gupta, M. (2017). Real-time IMU sensor fusion using complementary filter for tilt measurement. International Journal of Computer Science and Engineering,5(3), 22-28.

[19] Nayak, R., & Rao, P. (2021). Development of IMU-based attitude estimation system using Arduino and MPU6050. International Journal of Robotics and Control Systems, 2(4), 309-317.

[20] Singh, R., & Kumar, A. (2019). Interfacing MPU6050 with Arduino for motion tracking applications and display on LCD 16x2. International Journal of Advanced Research in Computer and Communication Engineering, 8(6), 45-50.